Pioneer Delivery Part III: Dynamic Programming Algorithm Report

Darien Chau

November 20, 2026

1 Introduction

The `pioneer` program solves three planning problems with dynamic programming: weighted interval scheduling (Task A), 0/1 knapsack with a fuel budget (Task B), and event-log cleanup that keeps a required pattern as a subsequence (Task C). The solvers are in `src/Scheduling.cpp`, `src/Knapsack.cpp`, and `src/Cleanup.cpp`. The code is C++20. The recorded build used GCC 14.2.0 and CMake 3.31.6.

Test IDs such as A-06 refer to rows in `docs/TestResults.md`. Each row lists the input, the expected value, the printed output, and the exit code.

2 Task A: maximum-profit delivery scheduling

2.1 Preprocessing: sorting and the predecessor $p(i)$

Deliveries are sorted by nondecreasing finish time. Ties are broken by start time, then by input position. The sorted deliveries are numbered $1, \ldots, n$, and index 0 is a sentinel. Let s_i, f_i, and v_i be the start, finish, and profit in integer cents of sorted delivery i.

The predecessor $p(i)$ is the largest index $j < i$ with $f_j \leq s_i$, or 0 if none exists. Because the finishes are nondecreasing, the indices in $1, \ldots, i-1$ with $f_j \leq s_i$ form a prefix. The code runs `std::upper_bound` on that prefix with key s_i, and the number of elements found is $p(i)$. Each predecessor costs $O(\log n)$, so all of them cost $O(n \log n)$.

2.2 DP state, base case, and recurrence

The state $OPT(i)$ is the maximum total profit of a compatible subset of the sorted deliveries $1, \ldots, i$. The base case is $OPT(0) = 0$. For $1 \leq i \leq n$,

$$OPT(i) = \max(OPT(i-1),\ v_i + OPT(p(i))).$$

The first term skips delivery i. The second term takes it. Once i is taken, the only earlier deliveries that remain compatible are those finishing by s_i, and those are exactly deliveries $1, \ldots, p(i)$.

2.3 Fill order and traceback

The table is filled for $i = 1, \ldots, n$. Both $OPT(i-1)$ and $OPT(p(i))$ are already known because $p(i) < i$.

The traceback starts at $i = n$. When $OPT(i) = OPT(i-1)$, delivery i is excluded and the walk moves to $i-1$. Otherwise delivery i is included and the walk moves to $p(i)$. Ties exclude. The collected IDs are reversed into chronological order. Before returning, the code checks that neighbouring selected deliveries do not overlap and that their profits sum to $OPT(n)$.

2.4 Complexity

Sorting costs $O(n \log n)$ time. The predecessor search costs $O(n \log n)$ time. The fill and traceback cost $O(n)$ time. The total time is $O(n \log n)$. The auxiliary arrays (sort order, starts, finishes, profits, p, and OPT) hold $O(n)$ entries. `std::sort` also uses $O(\log n)$ stack space.

2.5 Why earliest-finish greedy fails

The earliest-finish greedy repeatedly takes the compatible delivery that finishes first. It maximizes the number of deliveries, not the profit. The assignment example shows this. The earliest-finish schedule A, C, E earns 140.00, but the optimum B, D, F earns 180.00.

Test A-06 gives a small counterexample with three deliveries: A $[0, 2)$ 10.00, B $[0, 3)$ 100.00, and C $[2, 4)$ 10.00. Greedy takes A, which finishes at 2, and then C, which starts at 2 and is compatible with A. Its total is 20.00. Delivery B alone earns 100.00, and the solver returns 100.00 for A-06.

3 Task B: delivery selection with a fuel budget

3.1 DP state and base cases

The state $K(i, b)$ is the maximum profit of a subset of deliveries $1, \ldots, i$ whose total fuel is at most b, for $0 \le i \le n$ and $0 \le b \le B$. The base cases are $K(0, b) = 0$ for every b, because there are no deliveries, and $K(i, 0) = 0$, because every fuel cost is at least 1 and nothing fits in zero fuel. The table is one flat array of zeros, so both base cases hold without extra code.

3.2 Recurrence

Let $c_i \ge 1$ be the fuel cost and $v_i \ge 0$ the profit of delivery i. For $1 \le i \le n$ and $0 \le b \le B$:

$$K(i, b) = K(i-1, b) \quad \text{if } c_i > b,$$

$$K(i, b) = \max(K(i-1, b),\ v_i + K(i-1,\ b - c_i)) \quad \text{if } c_i \le b.$$

3.3 Fill order and traceback

The table is filled row by row: $i = 1, \ldots, n$, and within each row $b = 0, \ldots, B$. Each cell reads only row $i-1$, so no delivery is counted twice.

The traceback starts at (n, B). When $K(i, b) = K(i-1, b)$, delivery i is excluded and the walk stays in column b. Otherwise delivery i is included and the walk moves to $(i-1, b-c_i)$. Ties exclude. The code then sums fuel and profit over the selected deliveries, rejects duplicate IDs, and checks that the fuel is at most B and the profit equals $K(n, B)$.

3.4 Complexity and why the running time is pseudo-polynomial

The time is $O(nB)$, because the table has $(n+1)(B+1)$ cells and each costs $O(1)$. The space is $O(nB)$ for the table. The traceback and the selection flags cost $O(n)$.

The running time grows with the numeric value of B. The input stores B in about $\log_2 B$ bits, so a budget that takes a few digits to write can require on the order of B steps. That makes $O(nB)$ pseudo-polynomial, not polynomial in the input size.

For one delivery and $B = 999999999999$, the table would have 2×10^{12} cells. At 8 bytes per cell that is 16 TB. The solver refuses the input before allocating anything. It exits with status 1 and prints the message quoted in the README. The largest accepted table is 256 MiB, which is 33,554,432 cells. The test `KnapsackTests` exercises this boundary with budget 16777215 and one delivery, and it passes.

4 Task C: event-log cleanup with sequence alignment

4.1 DP state and impossible states

Let $n = |source|$ and $m = |pattern|$. Source position i (1-based) is eligible when its zero-based index $i-1$ is in `targetIndices`. The state $F(i, j)$ is the maximum number of deletions among the first i source characters, when the first j pattern characters are matched, in order, by kept source characters among those i. Every source character is either matched to a pattern character, deleted (only if eligible), or kept without matching.

$F(i, j)$ is unreachable when the pattern prefix of length j cannot be found inside the first i source characters. For example, $F(0, j)$ is unreachable for every $j > 0$. The code stores this state as -1. Valid scores are nonnegative, so -1 cannot be confused with a score, and the code never adds to it or compares it as a score.

The unreachable marker matters because treating it as 0 would be wrong. A state with value 0 would let a path claim that pattern characters were matched before any source character was read. The deletion count of that path would then be reported as valid even though the kept string may not contain the pattern. The marker makes every such path unavailable for extension.

4.2 Base cases and recurrence

The base cases are $F(0, 0) = 0$ and $F(0, j) = \perp$ for $j \geq 1$. The value $F(i, 0)$ is the number of eligible positions among the first i source characters. The skip transition gives this value without a separate formula.

For $1 \leq i \leq n$ and $0 \leq j \leq m$, let $e_i = 1$ if source position i is eligible and $e_i = 0$ otherwise. The two candidates are:

- Skip (source position i is not matched). If $F(i-1, j)$ is reachable, the candidate is $F(i-1, j) + e_i$. An eligible skipped character is deleted, and an ineligible one is kept.
- Match (source position i matches pattern position j). If $j \geq 1$, the characters are equal, and $F(i-1, j-1)$ is reachable, the candidate is $F(i-1, j-1)$. No deletion is credited, because the character is needed by the pattern.

$F(i, j)$ is the larger candidate, or $\perp$ if neither exists.

4.3 Fill order and traceback

The outer loop runs over $i = 1, \ldots, n$ and the inner loop over $j = 0, \ldots, m$. Row $i - 1$ is complete before row i reads it.

The traceback starts at (n, m). A skip is preferred whenever it reproduces $F(i, j)$. Otherwise the match must reproduce it. Each deleted eligible character is recorded by its original index. The list is reversed into ascending order. Before returning, the code checks that the count equals the optimum, that every index is eligible and in ascending order, and that the source with those positions removed still contains the pattern (a two-pointer check that does not use the table).

4.4 Example C-05

Take `abaaba`, pattern `aba`, and eligible indices $0, 1, 2$. The DP returns 3. It matches the pattern at positions 3, 4, and 5 and deletes positions 0, 1, and 2. The earliest alignment matches the pattern at positions 0, 1, and 2. That alignment uses all three eligible positions, so the greedy deletion count is 0. The DP is therefore not a greedy alignment.

4.5 Complexity

The time is $O(nm)$, because the table has $(n + 1)(m + 1)$ cells and each costs $O(1)$. The traceback costs $O(n + m)$. The auxiliary space is $O(nm)$ for the table, which uses 4-byte cells and is limited to 256 MiB, or 67,108,864 cells.

5 Testing

```
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Release
cmake --build build
ctest --test-dir build --output-on-failure
```

The CTest run passed 9 of 9 tests. Every named case in `docs/TestResults.md` matched its expected optimum, and every printed selection satisfied its validity check. Eight invalid inputs exited with status 2, and the table-limit input exited with status 1.

Program	Named cases	Seeds 1 to 25 (checks)	Result
SchedulingOracleTests	A-01 to A-08	66 of 66	self-check 8 of 8
KnapsackOracleTests	B-01 to B-08	66 of 66	self-check 8 of 8
CleanupOracleTests	C-01 to C-10	70 of 70	self-check 10 of 10

Table 1: Oracle programs. Each production comparison checks the optimum and the validity invariant.

The unit test executables report 1878 of 1878 checks (scheduling), 2181 of 2181 checks (knapsack), and 95741 of 95741 checks (cleanup, including a sweep of 15932 generated cases).

The oracles enumerate every subset: all subsets for A and B (at most 10 deliveries), and all subsets of the eligible indices for C (at most 12). They share no DP code with the solvers. They were written before the solvers were merged, and the worker that wrote them never opened the solver sources. The mutation test recorded in `docs/gates/G3.md` injected four bugs. Each bug was caught by at least one suite: a touching-interval boundary error in scheduling, item reuse in

knapsack, ignoring eligibility in cleanup, and an off-by-one compatibility error in scheduling. That mutation run was not repeated for this report.

The limits are that the oracles check small inputs only, and the mutation test covers four mutants, not every possible fault.

6 Attribution

- Claude Code with Claude Sonnet 5.5 acted as orchestrator. It read the assignment and the design handoff, created the repository and branches, dispatched and reviewed the workers, re-ran every build and test itself, ran a mutation test of the test suites, merged the branches, wrote the gate records, and made the Git commits.
- Kiro CLI with Claude Haiku 5.5 ran five worker sessions: W1 the CMake/CTest skeleton, input parsers, output formatter, command-line interface, and help; W2 Task A (scheduling); W3 Task B (knapsack); W4 Task C (cleanup); W5 the independent brute-force oracle programs, test fixtures, and documentation drafts.
- Verification performed: independent clean builds in a container (GCC 14.2, CMake 3.31) and on the host (GCC 16.2, CMake 4.4), 9 of 9 CTest tests, brute-force oracle comparison on named cases plus seeds 1 to 25, a four-bug mutation test, end-to-end CLI checks. The numbers come from `docs/gates/`.
- The design handoff documents (the specifications, tickets and worker briefs that the work followed) were prepared with GPT-6 (High reasoning setting), as reported by the project owner.
- Human contribution. The project owner is the original developer of Pioneer Delivery and is also the Part III owner. The owner set up the project and tools, planned the work, made the executive decisions, made the check-ins on the implementation, managed the final deliverables, and handled the tasks the agents could not do themselves.

7 Challenge and lesson learned

Sequence alignment, which is Task C, was the hardest part for me, and its mostly because I didn't know where to start. At first, my plan was to loop over the strings and delete what seemed appropriate. Most of my progress came from reading, speculation, and guesswork, not from solving it on my own. The tests passed, but I don't consider that the same as understanding it. Truthfully, I do a lot better on things when I have existing practice surrounding those issues beforehand. To do something that I don't feel confident in, my approach will most definitely show it.